

Ecommify

Diseño Conceptual y Lógico de Base de Datos Híbrida

PostgreSQL 16 + MongoDB 7.x · Dataset Olist 2018



9

ENTIDADES



99K

ÓRDENES



1M+

GEO-RECORDS



100K

RESEÑAS

EQUIPO

· Fredy Orlando Pulido Quintero · Myriam Andrea Martinez Fontecha · Nicolas Felipe Torres Amaya · Juan Francisco Javier Perez Rivero

<https://github.com/insoftcol/optimizacion-y-diseno-base-de-datos-v2/>

MARKETPLACE B2C2C

Vendedores independientes registran productos en un catálogo unificado; los clientes realizan pedidos con múltiples ítems y métodos de pago mixtos; tras la entrega emiten reseñas opcionales.

DATASET DE PRUEBAS

Brazilian E-Commerce de Olist (2016-2018), 9 archivos CSV con 99.441 órdenes, 112.650 ítems, 103.886 pagos, 100.150 reseñas y 1 millón de coordenadas geoespaciales. Representa el comportamiento real de la plataforma.

DOMINIO MODELADO



Customers

99K registros



Orders

Tabla de hechos

OrderItems

1:N por orden



Payments

Métodos mixtos



Reviews

41% nulos



Products

73 categorías



Sellers

Sin geo (alcance)



Geolocation

1M+ coords



AJUSTE DE ALCANCE

La relación seller → geolocation se elimina en esta fase. La geolocalización aplica solo a customers para cálculo de envíos cliente ↔ depósito.



REQUISITOS FUNCIONALES (RF)

- RF-01** Registro de clientes con datos personales y geolocalización
- RF-02** Catálogo con atributos variables por categoría (JSONB)
- RF-03** Procesamiento atómico de órdenes con múltiples ítems
- RF-04** Pagos con métodos múltiples y cuotas
- RF-05** Recepción de reseñas opcionales tras entrega
- RF-06** Cálculo de envío por distancia cliente↔depósito (PostGIS)
- RF-07** Búsqueda de productos tolerante a typos (pg_trgm)
- RF-08** Dashboards: ventas/categoría/mes, segmentos RFM
- RF-09** Promociones por rango temporal (TSTZRANGE)
- RF-10** Vista consolidada orden completa para BI (orders_summary)



REQUISITOS NO FUNCIONALES (RNF)

- RNF-01** OLTP p95 *< 200 ms inserción de orden*
- RNF-02** OLAP p95 *< 100 ms consulta MV*
- RNF-03** Disponibilidad *99.5 % uptime mensual*
- RNF-04** Consistencia *100 % FK válidas (pagos)*
- RNF-05** Escalabilidad *≥ 1.2 TPS sostenidos*
- RNF-06** RTO *< 30 s failover PG*
- RNF-07** RPO *0 transacciones perdidas*
- RNF-08** Seguridad *bcrypt cost 12 + pgp_sym*
- RNF-09** Mantenibilidad *DDL versionado en Git*
- RNF-10** Free tier *≤ 500 MB Supabase + 512 MB Atlas*

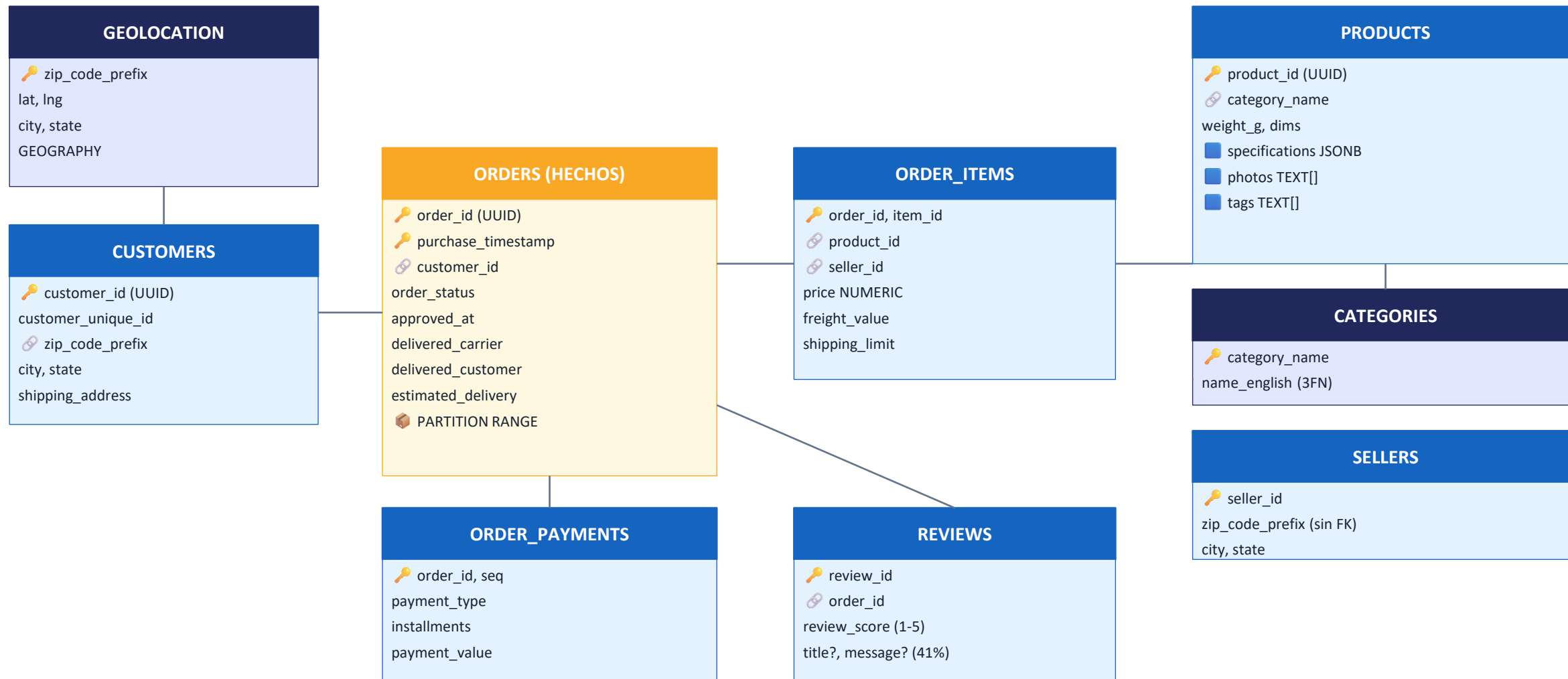


TABLA	PK / Tipo	Columnas clave	Tipos avanzados
customers	UUID	customer_unique_id, zip, state	address (composite)
sellers	UUID	zip, city, state (sin FK geo)	—
products	UUID	category, dims, weight	JSONB, TEXT[], composite
geolocation	INTEGER	lat, lng, city, state	GEOGRAPHY(POINT,4326)
orders	UUID + timestamp	status, purchase_ts, delivered	JSONB audit_log · PARTITION RANGE
order_items	compuesta (3)	product_id, seller_id, price	NUMERIC(10,2)
order_payments	compuesta (3)	type, installments, value	CHECK domain
reviews	UUID	score, title?, message? (41%)	TEXT (nullable)
promotions	UUID	product_id, discount_pct	TSTZRANGE + EXCLUDE GIST

📌 Todas las tablas con `updated_at` gestionado por trigger reusable `set_updated_at()`

JSONB

products.specifications

73 categorías con atributos heterogéneos. Schema flexible evita columnas NULL masivas.

```
specifications JSONB
+ índice GIN jsonb_path_ops
specs @> '{"resolution":"4K"}'
```

ARRAY

products.photos / tags

URLs ordenadas y etiquetas. Operadores @>, && con índice GIN. Sin tabla intermedia.

```
photos TEXT[]
tags TEXT[]
WHERE tags && ['samsung','lg']
```

TSTZRANGE

promotions.period

Ventana de promoción. Operador @> con índice GIST. EXCLUDE previene solapamientos.

```
period TSTZRANGE
EXCLUDE USING GIST (product_id WITH =, period WITH &&)
```

COMPOSITE

address, dimensions

Tipos reutilizables entre tablas. Encapsula múltiples campos en una columna.

```
CREATE TYPE address AS (
  street TEXT, city VARCHAR(60),
  state CHAR(2), zip INT, country CHAR(2))
```



PostGIS 3.4

Cálculo geoespacial sobre GEOGRAPHY

Distancia cliente↔depósito en metros usando ST_Distance con índice GiST

```
ST_DWithin(c.location, w.location, 50000)
```



pg_trgm

Búsqueda fuzzy con trigramas

Productos tolerantes a typos: 'samsung' encuentra 'samsung' con índice GIN

```
WHERE product_category_name % 'celular'
```



pgcrypto

Hashing y cifrado simétrico

bcrypt cost 12 para passwords + pgp_sym_encrypt para campos sensibles

```
crypt(password, gen_salt('bf', 12))
```



btree_gin

Índices GIN multi-columna mixtos

Combinar JSONB + escalar: specs @> X AND category = Y en una sola búsqueda

```
CREATE INDEX ... USING GIN (specs, category)
```

PROYECCIÓN
ETL

orders_summary

Vista analítica desnormalizada de la orden completa

- `_id` = `order_id` de PostgreSQL
- Embedded: `customer{}`, `items[]`
- Shard key: `hashed(_id)`
- ETL nightly + incremental



DESDE CSV

reviews

Reseñas con esquema flexible (41% nulos)

- `comment_title?` opcional
- `comment_message?` opcional
- Índice text para búsqueda
- Sin ETL ongoing



ETL ← PG

products_catalog

Catálogo desnormalizado con category embebida

- `_id` = `product_id` de PG
- `category { name_pt, name_en }` embebido
- Sin JOIN en lectura
- Refresh upsert on change



DESDE CSV

geolocation

Coords geoespaciales con índice 2dsphere

- `location`: GeoJSON Point
- 1M+ documentos
- Índice 2dsphere nativo
- Consultas `near`, `geoWithin`



Matriz de Decisión por Entidad

Asignación PostgreSQL vs MongoDB con criterios cuantitativos

Entidad	% Nulos	FK	Consist.	Vol. Lect.	Esq. Flex.	Motor	CAP
orders	2.7%	A	A	M	B	PostgreSQL	CP
order_items	0%	A	A	M	B	PostgreSQL	CP
order_payments	0%	A	A	B	B	PostgreSQL	CP
customers	0%	A	M	M	B	PostgreSQL	CP
sellers	0%	A	M	B	B	PostgreSQL	CP
reviews	41.1%	B	B	A	A	MongoDB	AP
geolocation	0%	B	B	A	B	MongoDB	AP
products	0.8%	M	M	A	A	Mixta PG+MDB	CP/AP
orders_summary	N/A	B	B	A	A	MongoDB	AP

A = Alto · M = Medio · B = Bajo · Criterios: % nulos, FK estricta, consistencia, volumen de lectura, esquema flexible

TEOREMA CAP (Brewer, 2000)

Imposible garantizar Consistencia + Disponibilidad + Tolerancia a Particiones simultáneamente. Se elige entre sistemas CP y AP, ya que P es inevitable.



PostgreSQL 16 — Sistema CP

Consistencia + Tolerancia a Particiones

Garantiza ACID completo para datos transaccionales. Sacrifica disponibilidad durante failover (~30 s). Una orden duplicada = error financiero irreversible.

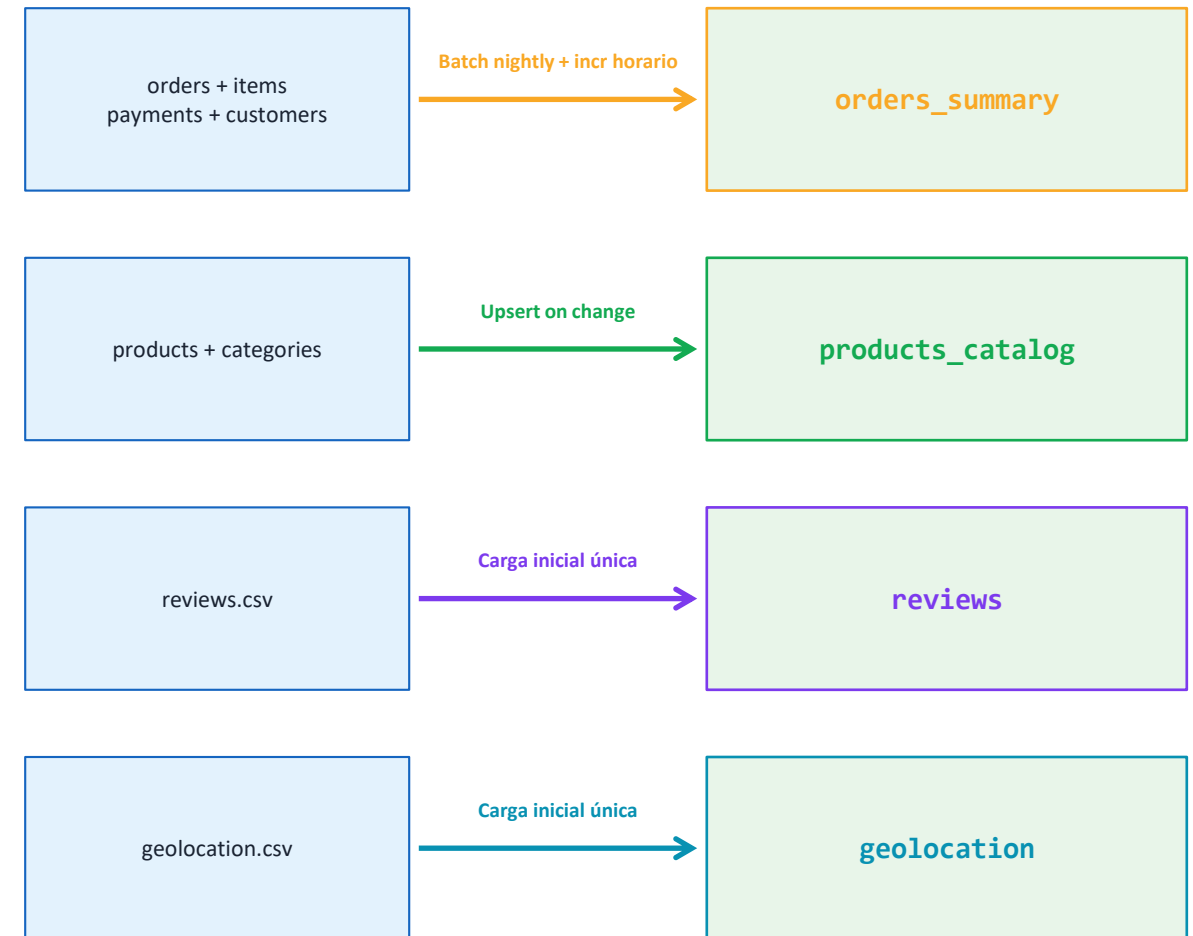


MongoDB 7.x — Sistema AP

Disponibilidad + Tolerancia a Particiones

Prioriza disponibilidad y escalabilidad horizontal. Consistencia eventual en réplicas. Una reseña con 30 s de retraso no es un error de negocio.

INTEGRACIÓN HÍBRIDA — 4 FLUJOS ETL



Limitaciones y Mitigaciones

Trade-offs identificados y cómo se compensan



LIMITACIÓN

500 MB en Supabase free

IMPACTO

No cabe dataset completo + tablas adicionales

MITIGACIÓN

Muestra de geolocation (100K registros) en Supabase. Dataset full en Docker local. Migrar a Pro al superar.



LIMITACIÓN

Sharding no disponible en Atlas M0

IMPACTO

No se puede escalar horizontalmente en plan gratuito

MITIGACIÓN

Sharding emulado en cluster Docker local. Migración a Atlas M10 (\$60/mes) cuando crezca volumen real de producción.



LIMITACIÓN

Sin FK nativa MongoDB ↔ PostgreSQL

IMPACTO

order_id puede quedar huérfano en reviews

MITIGACIÓN

Política ON DELETE RESTRICT en PG impide borrar orders con dependencias. Soft-delete en su lugar. Validación de integridad en aplicación.



LIMITACIÓN

Latencia ETL para orders_summary

IMPACTO

Datos en MongoDB están 1-24 h desactualizados

MITIGACIÓN

Aceptable para dashboards ejecutivos. Consultas en tiempo real (checkout, perfil) van directo a PG. ETL incremental horario.



LIMITACIÓN

Lecturas secundarias MDB con lag

IMPACTO

Cliente puede ver review propia con retraso de segundos

MITIGACIÓN

readConcern: majority para campos críticos. readPreference: primary cuando importa la consistencia inmediata.

Plan de Implementación

FASE 1

Carga inicial

Semana 1

- Setup Docker local
- Aplicar 8 scripts DDL
- Cargar CSVs Olist
- Validar integridad

FASE 2

Tipos avanzados

Semana 2

- JSONB specifications
- PostGIS warehouses
- Promociones TSTZRANGE
- Triggers updated_at

FASE 3

OLAP + ETL

Semana 3

- Materialized views
- ETL PG → MongoDB
- Índices 2dsphere
- Sharding emulado

FASE 4

Validación final

Semana 4

- Benchmark OLTP/OLAP
- Stress test cargas
- Refinamiento queries
- Documentación



Repositorio GitHub

<https://github.com/insoftcol/optimizacion-y-diseno-base-de-datos-v2/>

29 archivos · 8 DDL scripts · 2 ETL Python · 4 ejemplos MongoDB · diccionario TSV